

基于本地知识库的论文搜索助手系统 — 实验报告

课程: 数据库系统实验

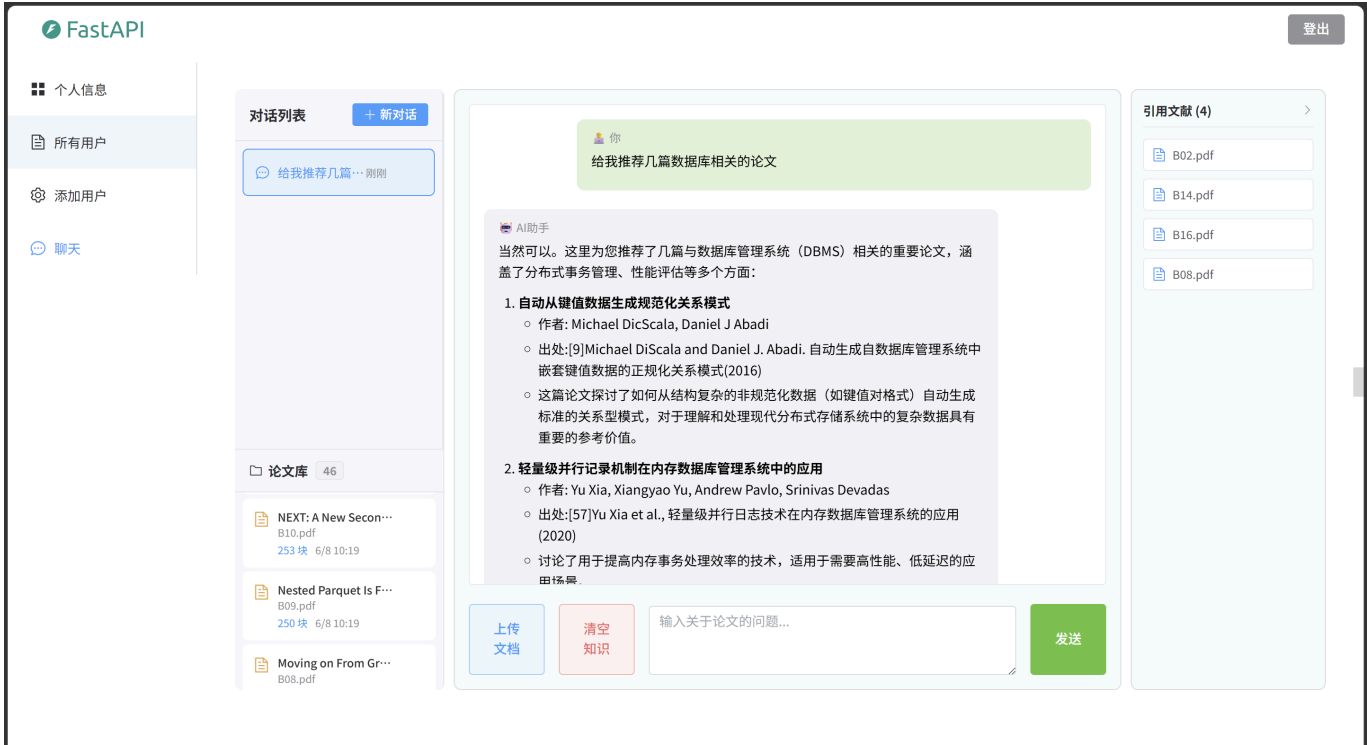
完成日期: 2026年6月8日

一、项目概述

本项目开发了一个基于本地知识库的论文搜索助手系统，采用前后端分离 + 微服务架构。前端使用 Vue 3 + TypeScript，主后端和算法层分别使用独立的 FastAPI 服务，向量数据库选用 ChromaDB，大语言模型通过 Ollama 调用 Qwen 系列模型。系统支持论文上传与解析、基于 RAG（检索增强生成）的智能问答、流式对话输出、文献来源溯源以及用户认证等核心功能。

项目经历了三个版本的迭代开发：v1.0 为基础版本，已通过助教现场验收；v1.1 新增了聊天记录持久化、多会话管理并优化了 RAG 管线；v1.2 新增了论文库管理、标题自动提取，并对前端鉴权体系进行了全面加固。以下先对 v1.0 的基础工作做简要介绍，再重点说明 v1.1 和 v1.2 的增量工作。

最终效果展示：



二、v1.0 基础版本概述（已验收）

2.1 系统架构

系统由四个独立运行的服务组成：

服务	端口	技术栈	职责
前端	8080	Vue 3, TypeScript, Vite, Element Plus	页面展示与用户交互
主后端	8000	FastAPI, SQLAlchemy, SQLite, PyJWT	用户认证、接口转发、安全控制

服务	端口	技术栈	职责
算法层	8001	FastAPI, ChromaDB, Ollama	文档处理、向量化、检索、对话生成
向量库	8002	ChromaDB	文本向量的持久化存储与检索

前端只与主后端通信，主后端将涉及 AI 能力的请求转发给算法层，算法层直接与 ChromaDB 和 LLM 交互。这种分层设计使各模块职责清晰、易于扩展。

2.2 已实现的核心功能

v1.0 版本实现了以下完整功能链路：

用户认证：注册与登录、密码哈希存储、JWT Token 验证，认证后才能访问聊天与上传功能。

文档管理：支持 PDF 和 Word 文件上传，通过 PDFPlumber 和 python-docx 进行文本提取与清洗，将文本分块后调用阿里云 Embedding API 向量化，存入 ChromaDB。

RAG 检索与对话：用户提问后，系统将问题向量化并在 ChromaDB 中检索 Top-N 相似文本块，经 Qwen3-Rerank 精排后选出最相关的片段，将其作为上下文注入 Prompt，调用 Qwen 大模型生成回答。整个过程以 SSE 流式传输，前端逐字展示。

前端展示：消息区分用户与 AI 样式，支持 Markdown 渲染和 KaTeX 数学公式，回答末尾显示引用的论文来源列表（支持预览和下载）。

知识库管理：支持一键清空知识库中所有向量和上传文件。

2.3 v1.0 开发中解决的主要问题

在 v1.0 开发过程中解决了若干工程问题：登录后 Token 刷新丢失（改为本地存储持久化）、前后端接口契约不一致（统一数据结构）、Markdown 表格和公式渲染异常（切换兼容的 KaTeX 插件）、文件上传跨域与请求头冲突（让浏览器自动管理 multipart 头）、以及初版架构耦合过紧（拆分主后端与算法层）等。

三、v1.1 版本更新（重点）

v1.1 围绕"聊天记录持久化与多会话管理"这一核心需求展开，同时修复了多个后端和前端 Bug，并对 RAG 管线进行了诊断与优化。本次更新涉及 15 个文件的改动（其中 2 个新增文件、1 个重写文件）。

3.1 聊天记录持久化与多会话管理

背景与动机：v1.0 的聊天记录仅存在于前端内存中，刷新页面即丢失，且不支持多会话切换。这在实际使用中严重影响体验。

设计决策：选择将聊天记录持久化到 MySQL 数据库（而非 localStorage），理由是数据库方案支持跨设备同步、不受浏览器缓存清理影响，且便于后端统一管理。会话标题取自用户首条消息的前 20 个字符，为空时兜底为"新对话"。

后端改动：

在数据模型层面，models.py 新增了 Conversation 表和 Message 表。Conversation 包含 id、user_id（外键关联用户表）、title、created_at、updated_at 等字段；Message 包含 id、conversation_id（外键关联会话表）、

role、content（使用 MEDIUMTEXT 类型以支持超长消息）和 created_at。schemas.py 新增了对应的 6 个 Pydantic Schema（会话创建、响应、列表、详情以及消息响应、创建），并为 ChatRequest 增加了 conversation_id 可选字段。crud.py 新增了 6 个 CRUD 函数，包括创建会话、按用户分页查询会话列表（按更新时间倒序）、获取会话详情、删除会话（先删消息再删会话以保证外键约束）、更新会话标题等。其中 create_message 函数在写入消息时手动更新会话的 updated_at 字段，因为 SQLAlchemy 的 onupdate 在某些场景下不会自动触发。

在接口层面，main.py 新增了 4 个 API 端点：GET /conversations/（分页查询会话列表）、GET /conversations/{id}（获取会话详情及所有消息）、DELETE /conversations/{id}（删除会话及其所有消息）、POST /conversations/{id}/messages（手动追加消息）。最关键的是对 POST /chat/stream 的改造——流式对话接口的处理流程变为：自动创建会话 → 保存用户消息 → 发送 meta SSE 事件（携带新会话 ID）→ 转发请求到算法层 → 流式累积助手回复内容 → 流结束后用独立的 SessionLocal() 实例保存完整的助手回复。这里使用独立 session 是为了避免在 SSE 生成器中持有已关闭的数据库连接。

前端改动：

新增了 store/conversation.ts，这是一个完整的 Pinia 状态管理模块，管理会话列表、当前会话 ID、当前消息列表和加载状态，提供加载会话列表、切换会话、新建会话、设置会话 ID、添加本地消息、更新最后一条消息内容与来源、删除会话（删除后从服务端重新拉取完整列表以确保一致性）、前置新会话（带去重检查）等方法。

新增了 ConversationList.vue 组件，展示会话列表（标题、人性化时间格式如“刚刚”、“X分钟前”），支持新建、切换和删除操作。

Chat.vue 被重写为三栏布局：左侧会话列表侧栏（200px）+ 中央聊天区域（flex:1）+ 右侧参考资料侧栏。流式解析中处理 meta 事件获取 conversation_id，并实现了兜底机制——当 meta SSE 事件因 chunk 边界切割未被解析时，流结束后从服务端拉取最新会话。

ReferenceSidebar.vue 改为可折叠设计：收起状态仅 36px 宽（显示图标和来源数量），展开状态 180px（完整论文列表），默认收起以节省聊天区域空间。

3.2 Bug 修复

v1.1 修复了一系列影响系统稳定性的 Bug：

后端方面，schemas.py 中 ConversationDetail 引用了尚未定义的 MessageResponse 导致启动时 NameError，通过调整定义顺序修复；database.py 中 SQLAlchemy 2.0 弃用了 sqlalchemy.ext.declarative.declarative_base 的导入路径，改为从 sqlalchemy.orm 导入；crud.py 中 create_message 存在多余的 db.commit() 调用，改为统一单次提交；消息内容字段从标准 Text（64KB 限制）升级为 MEDIUMTEXT（16MB），防止长对话消息被截断。

前端方面，标题截取逻辑误用了 Python 的 .strip() 方法（JavaScript 中不存在），修复为 .trim()；SSE 处理中收到 [DONE] 信号时使用 break 可能跳过同一 chunk 中的后续事件，改为 continue。

用户反馈的 UI 问题也做了修复：导航栏在聊天页面被条件隐藏（v-if="!isChatPage"），用户要求始终可见，移除了条件渲染；每条消息误创建新对话的问题通过增加 meta 事件兜底机制解决；删除会话后乐观过滤与服务端状态不一致，改为删除后重新拉取完整列表。

3.3 RAG 检索流程诊断与优化

针对用户反馈的"AI 回答中提到参考资料为空"问题，对 RAG 管线进行了全链路排查，逐层测试了 ChromaDB 连通性（8071 个文本块）、阿里云 Embedding API（1024 维向量）、ChromaDB 向量检索、阿里云 Rerank API、LLM 响应，以及端到端全链路。测试结论为管线整体功能正常，"参考资料为空"可能是特定查询匹配度低或服务临时不可用导致。

在此基础上进行了代码优化：原 `retrieve_knowledge()` 的异常处理会静默吞掉所有错误（仅打印一行日志），现新增 `traceback.print_exc()` 输出完整堆栈；`handle_rag_chat_stream` 中新增 `[RAG]` 日志行，每次请求打印 query 摘要、context 长度和 sources 信息，便于实时监控 RAG 是否命中；`rerank_service.py` 同样新增了 `traceback` 日志。

四、v1.2 版本更新（重点）

v1.2 围绕"论文库精细化管理"和"前端鉴权加固"两大方向展开，同时新增了论文标题自动提取和批量上传工具，并对 RAG 检索参数进行了调优。本次更新涉及 20 个文件的改动（其中 4 个新增文件、1 个重写文件）。

4.1 论文库管理

背景与动机：v1.0 和 v1.1 的知识库管理只支持"全部清空"操作，无法查看已上传的论文列表，也无法单独删除某篇论文。这在论文数量增多后极不方便。

后端改动：

`models.py` 新增 `Paper` 表，字段包括 `id`、`filename`（唯一索引，防止重复入库）、`title`、`chunks_count`（知识块数量）和 `uploaded_at`。`crud.py` 新增 5 个 `Paper` CRUD 函数。`schemas.py` 新增 `PaperResponse` 和 `PaperListResponse` 两个 Schema。

`main.py` 新增了 2 个 API 端点：`GET /papers/`（查询论文列表）和 `DELETE /papers/{paper_id}`（删除单篇论文）。删除操作的流程为：先从 MySQL 查出论文记录 → 转发给算法层删除 ChromaDB 中对应向量 → 确认算法层删除成功后再删 MySQL 记录（保证数据一致性）。文件名做了 URL 编码（`urllib.parse.quote()`）以防止中文路径出错。同时改造了 `POST /upload`（上传成功后将论文信息写入 MySQL）和 `DELETE /clear`（清空时同步删除 `papers` 表所有记录）。

算法层的 `knowledge_service.py` 新增了 `delete_paper_by_filename()` 函数，通过 ChromaDB 的 `where={"source": filename}` 过滤器定位该论文的所有向量并批量删除，同时删除 `uploads` 目录下的物理文件。`main.py` 新增 `DELETE /internal/papers/{filename}` 内部接口供主后端调用。

前端改动：

`api.ts` 新增 `GetPapers()` 和 `DeletePaper()` 两个 API 调用函数。全新的 `PaperList.vue` 组件展示论文库列表（标题、文件名、知识块数量、上传时间），支持 hover 后点击删除按钮进行单篇删除（带 Element Plus 确认弹框防止误操作），通过 `defineExpose` 暴露 `loadPapers()` 方法供父组件在上传成功或清空后自动刷新列表。`Chat.vue` 的布局进一步调整为左侧栏 = 对话列表 + 论文库面板的组合形式。`ChatInput.vue` 新增 `upload-success` 事件用于通知父组件上传完成。

4.2 论文标题自动提取

背景：系统原先只存储文件名（如 `B00.pdf`），在论文库列表中无法直观识别论文内容。

`parser.py` 新增了 `extract_title()` 统一入口函数，采用多策略级联方案：对于 PDF，优先读取文档元数据的 `Title` 字段（大部分英文论文都包含此字段）；对于 DOCX，优先读取 `core_properties.title`；若元数据

中无标题，则从首页文本中提取最长的有意义行（内置噪声过滤器，跳过页码、卷期号、会议名等无关行）；最终兜底使用去掉扩展名的文件名。

`knowledge_service.py` 的 `process_and_ingest_document()` 在入库流程末尾调用 `extract_title()`，将标题加入返回结果。主后端从算法层响应中取出 `title`，一并存入 MySQL。前端以标题为主显示名，文件名作为次要信息（仅当标题与文件名不同时显示）。

由于 `papers` 表是在加 `title` 字段之前创建的，SQLAlchemy 的 `create_all` 不会自动加列，因此需要手动执行 `ALTER TABLE papers ADD COLUMN title VARCHAR(1024) DEFAULT '' AFTER filename` 进行数据库迁移。

4.3 RAG 检索参数调优

将 `retrieval_service.py` 中的 `final_top_k` 从 2 调整为 5。理由是每个 chunk 约 400 字符，5 块约 2000 字符（约 800~1000 token），对 qwen2.5:7b 的上下文窗口来说既充分又不会引入过多噪声。粗排仍保持 100 条候选，精排通过 Qwen3-Rerank 从中选出 Top 5。

4.4 批量上传脚本

新增了 `batch_upload.py` 和 `batch_upload.bat` 两个工具文件。Python 脚本自动登录获取 JWT Token，然后逐个上传指定目录下的所有 PDF/DOCX 文件，每篇显示处理进度、知识块数量和提取到的标题，最后汇总成功/失败数和总耗时。Windows 批处理文件作为启动器，双击即可运行（账号密码内置于脚本配置区）。

4.5 前端鉴权体系加固

背景与动机：v1.0 的鉴权仅在后端进行（返回 401），前端没有任何拦截机制。用户直接访问 `localhost:5174/index/chat` 即可看到界面内容，尽管后端请求会被拒绝，但界面展示本身就不应可见。此外 Store 刷新页面后丢失登录态，退出登录的逻辑也有 Bug（将 token 设为字符串 `'token'` 而非清除，该值为 `truthy` 导致退出后仍被识别为登录状态）。

user store 完全重写：`store/user.ts` 从初始化时即从 `localStorage` 恢复 token 和 `userName`（解决刷新掉登录的问题），新增 `isAuthenticated` getter 判断登录状态，新增 `setAuth()` 统一写入登录态，新增 `logout()` 统一清除状态并跳转登录页（通过动态 `import()` 引入 `router` 避免循环依赖）。

路由守卫：`router/index.ts` 为 `/index` 路由（含 `/chat` 等所有子路由）加了 `meta: { requiresAuth: true }`，登录页和注册页加了 `meta: { guestOnly: true }`。新增全局 `beforeEach` 守卫：未登录访问受保护页面时跳转登录页；已登录访问登录/注册页时跳转主页。

HTTP 拦截器：`http.ts` 的响应拦截器新增 401 处理逻辑——后端返回 401 时自动调用 `userStore.logout()` 清除登录态并跳转。请求拦截器的 token 判断从 `!== 'token'` 改为 `truthy` 检查（适配新的默认值 `''`）。对于使用原生 `fetch` 的 `ChatWithLLMStream`（绕过了 `axios` 拦截器），单独加了 401 状态码判断和 `logout` 调用。

Bug 修复：`Header.vue` 的退出登录逻辑从直接赋值为字符串 `'token'` 改为调用 `userStore.logout()` 走统一清除流程。`LoginForm.vue` 简化登录逻辑，用 `userStore.setAuth()` 替代手动赋值和手动 `localStorage` 操作。

4.6 其他修复

修复了上传接口参数顺序导致的 Python `SyntaxError`（有默认值的参数不能排在无默认值参数前面）。修复了删除接口未做 URL 编码导致中文文件名请求失败的问题。修复了删除接口错误处理中的逻辑缺陷——原实现中算

法层删除失败仍会删掉 MySQL 记录导致数据不一致，改为先确认算法层删除成功再删 MySQL 记录。修复了清空接口缺少 JWT 鉴权依赖的问题。

五、工作量总结

5.1 文件改动统计

版本	新增文件	重写文件	修改文件	合计
v1.0	全部文件均为新增（首次开发）	—	—	~20+
v1.1	2（conversation.ts, ConversationList.vue）	1（Chat.vue）	12	15
v1.2	4（PaperList.vue, batch_upload.py, batch_upload.bat 等）	1（user.ts）	15	20

5.2 功能增量对比

功能维度	v1.0	v1.1	v1.2
聊天记录	仅内存，刷新丢失	MySQL 持久化	保持
会话管理	不支持	多会话新建/切换/删除	保持
论文库管理	仅支持全部清空	保持	列表查看、单篇删除
论文标题	仅显示文件名	保持	自动提取元数据标题
前端鉴权	无前端拦截	保持	路由守卫 + 401 拦截 + 状态持久化
RAG 参数	final_top_k = 2	增加诊断日志	final_top_k = 5
批量工具	无	无	batch_upload 脚本
布局	双栏	三栏（+会话列表）	三栏 + 论文库面板

5.3 技术亮点

全栈数据一致性保障：论文删除操作采用"先算法层后数据库"的事务顺序，会话删除采用"先消息后会话"的外键约束顺序，消息写入时手动维护更新时间——这些细节体现了对分布式系统数据一致性的理解。

SSE 流式通信的健壮性设计：meta 事件传递会话 ID 并实现兜底机制（当 chunk 边界导致 meta 事件未被解析时自动从服务端重新拉取），[DONE] 信号使用 `continue` 而非 `break` 避免丢失同 chunk 内的后续事件。

多策略级联的标题提取：从文档元数据 → 首页文本智能提取 → 文件名兜底的三级策略，覆盖了不同格式和质量论文的场景。

完整的前端鉴权闭环：路由守卫（访问前拦截）→ 请求拦截器（附加 Token）→ 响应拦截器（401 自动登出）→ 统一退出逻辑，形成了从页面访问到 API 调用到登出操作的全链路覆盖。

六、开发总结与反思

整个项目经历了从基础功能搭建到逐步完善的过程。v1.0 建立了完整的 RAG 问答系统骨架，v1.1 解决了聊天记录无法持久化的核心体验问题并增加了多会话管理能力，v1.2 则在论文管理精细度和系统安全性两个维度上做了进一步提升。

在开发过程中，有几个教训值得总结。一是数据模型设计需要前瞻——v1.0 没有预留 Paper 表导致 v1.2 需要手动执行 ALTER TABLE 迁移；二是前端状态管理不能依赖内存——v1.0 和 v1.1 早期都因为刷新丢失状态而被迫改造；三是异常处理不能静默吞错——v1.1 的 RAG 诊断过程中发现原代码的 `except: pass` 模式导致问题排查极其困难，改为打印完整堆栈后效率大幅提升。

后续可进一步改进的方向包括：混合检索（向量 + 关键词 BM25）、多模态 PDF 内容提取（图表识别）、历史会话搜索、以及企业级部署方案（Docker 容器化 + 负载均衡）。